

SYNRX (Speak Your Next Round Experience)

A Project Report submitted in partial fulfilment of the requirements for the award of the degree of

Master of Computer Applications

By

Navdeep Verma	2484200124
Sushant Pathak	2484200194
Ritesh Kumar	2484200161

Group No. 4

Under the Guidance of

Assistant Professor Mr. Rohit Agarwal

Department of Computer Engineering & Applications

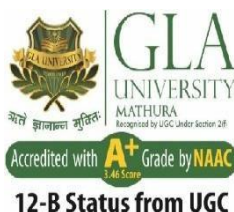
Institute of Engineering & Technology



GLA University

Mathura – 281406, INDIA

April, 2026



Department of Computer Engineering and Applications
GLA University, Mathura
17km Stone, NH-2, Mathura-Delhi Road, P.O.– Chaumuhan,
Mathura 281406 (U.P.), India

Declaration

I hereby declare that the work which is being presented in the MCA Project “**SYNRX (Speak Your Next Round Experience)**”, in partial fulfillment of the requirements for the award of the ***Master of Computer Applications*** and submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is an authentic record of my own work carried under the supervision of **Mr. Rohit Agarwal, Assistant Professor.**

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign _____

Name of Student: Sushant Pathak
University Roll No.: 2484200194

Sign _____

Name of Student: Navdeep Verma
University Roll No.: 2484200124

Sign _____

Name of Student: Ritesh Kumar
University Roll No.: 2484200161

Certificate

This is to certify that the above statements made by the candidate are correct to the best of my/our knowledge and belief.

Supervisor

(Mr. Rohit Agarwal)

Assistant Professor

Dept. of Computer Engg, & App.

Project Co-ordinator

(Dr. Sachendra Singh Chauhan)

Assistant Professor

Dept. of Computer Engg, & App.

Program Co-ordinator

(Mr. Siddharth Singh)

Assistant Professor

Dept. of Computer Engg, & App.

Date:

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude **Mr. Rohit Agarwal**, my project supervisor, for his invaluable support and guidance throughout the duration of this project.

His expertise and mentorship have been instrumental in shaping this work and navigating through its challenges.

I am also thankful to the faculty members of GLA University for their continuous encouragement and valuable feedback during the course of this project.

Sign _____

Name of Student: Sushant Pathak
University Roll No.: 2484200194

Sign _____

Name of Student: Navdeep Verma
University Roll No.:2484200124

Sign _____

Name of Student: Ritesh Kumar
University Roll No.: 2484200161

ABSTRACT

The growing competitiveness in campus placements has increased the need for a structured, personalized, and technology-driven preparation system for students. The project titled “SYNRX – Placement Preparation System” is designed to provide an integrated platform that helps students enhance their interview and aptitude skills through targeted practice, analytics, and real-time performance tracking.

SYNRX offers a comprehensive set of features, including real interview questions, topic-wise practice modules, timed mock tests, performance analytics, and progress dashboards. Users can practice frequently asked questions, analyse their strengths and weaknesses, and focus on areas that require improvement. The system also includes collaborative features that allow students to interact, discuss problems, and learn together, making preparation more effective and engaging.

The platform is built with a modern, responsive interface that ensures accessibility across devices. It incorporates secure authentication, structured content flow, and optimized data handling to provide a seamless experience. By combining structured learning, analytics-driven insights, and user-friendly design, SYNRX aims to improve students’ confidence, readiness, and overall placement performance.

This project demonstrates how technology can be effectively used to support students in preparing for competitive recruitment processes by offering personalized learning, consistent practice, and measurable outcomes.

TABLE OF CONTENTS

Declaration	ii
Acknowledgement	iii
Abstract	iv
List of Figures Tables	v
List of Figures	ix
Chapter 1: Introduction	1-3
1.1 Overview and Motivation	1
1.2 Objectives	2
1.3 Summary of Similar Systems	2
1.4 Organization of the Report	3
Chapter 2: Software Requirement Analysis	4-14
2.1 Requirement Analysis	4
2.1.1 Functional Requirements	4
2.1.2 Non-Functional Requirements	5
2.2 Feasibility Analysis	5
2.2.1 Technical Feasibility	5
2.2.2 Economic Feasibility	6
2.2.3 Operational Feasibility	6
2.2.4 Time Feasibility	6
2.3 Module Description	6
2.3.1 Student Module	7
2.3.2 Admin Module	7
2.3.3 SuperAdmin Module	8
2.4 Use Case Description	10
2.4.1 Student Use Case	10

2.4.2 Admin Use Case	11
2.4.3 SuperAdmin Use Case	13
2.5 System Architecture	14
Chapter 3: System Design	15-23
3.1 Data Flow Diagrams	15
3.1.1 Level 0 DFD	15
3.1.2 Level 1 DFD	16
3.1.3 Level 2 DFD	17
3.2 UML Diagrams	17
3.2.1 Class Diagram	18
3.2.2 Sequence Diagram	19
3.2.3 Object Diagram	20
3.2.4 Collaboration Diagram	21
3.3 ER Diagram	21
Chapter 4: Implementation	24-31
4.1 Technologies Used	24
4.1.1 Frontend Technologies	24
4.1.2 Backend Technologies	25
4.2 System Modules	25
4.2.1 Student Module	25
4.2.2 Admin Module	26
4.2.3 SuperAdmin Module	27
4.3 Backend Implementation	27
4.3.1 REST API Development	27
4.3.2 Database Implementation	28
4.3.3 Authentication & Security	28
4.3.4 Business Logic	29
4.4 Frontend Implementation	29
4.4.1 User Interface Development	29

4.4.2 Component Based Design	29
4.4.3 Communication with Backend	30
4.4.4 Displaying Analytics	30
4.5 Integration	30
4.5.1 API Integration	30
4.5.2 Module Integration	31
4.5.3 Testing & Debugging	31
4.5.4 Deployment	31
 Chapter 5: Testing & Results	 32-43
5.1 Testing Methods	32
5.1.1 Unit Testing	32
5.1.2 Integration Testing	32
5.1.3 Functional Testing	33
5.1.4 User Interface (UI) Testing	33
5.1.5 Performance Testing	33
5.1.6 Security Testing	33
5.2 Test Cases	34
5.3 Results & Screenshots	36
 Chapter 6: Conclusion	 44
Chapter 7: Summary	45
 APPENDICES	 46-47
Appendix: Geo-tagged photographs with project advisor	

LIST OF TABLES

1. Table 5.2.1 – Login Test Case	34
2. Table 5.2.2 – Invalid Login Test Case	34
3. Table 5.2.3 – Quiz Attempt Test Case	34
4. Table 5.2.4 – View Analytics Test Case	35
5. Table 5.2.5 – Faculty Create Quiz Test Case	35
6. Table 5.2.6 – SuperAdmin Add Faculty Test Case	35
7. Table 5.2.7 – Unauthorized Access Test Case	36

LIST OF FIGURES

1. Figure 2.4.1 – Use Case Diagram of User	11
2. Figure 2.4.2 – Use Case Diagram of Admin	12
3. Figure 2.4.3 – Use Case Diagram of SuperAdmin	13
4. Figure 2.5 – Sequence Diagram for Login	14
5. Figure 3.1.1 – Level 0 Data Flow Diagram	15
6. Figure 3.1.2 – Level 1 Data Flow Diagram	16
7. Figure 3.1.3 – Level 2 Data Flow Diagram	17
8. Figure 3.2.1 – Class Diagram	18
9. Figure 3.2.2 – Sequence Diagram	20
10. Figure 3.2.3 – Object Diagram	20
11. Figure 3.2.4 – Collaboration Diagram	21
12. Figure 3.3 – Er Diagram	22
13. Figure 5.3.1 – Login Page	36
14. Figure 5.3.2 – Student Dashboard	37
15. Figure 5.3.3 – Quiz Listing Page	37
16. Figure 5.3.4 – Quiz Interface (Question Page)	38
17. Figure 5.3.5 – Quiz Result Page	38
18. Figure 5.3.6 – Analytics / Performance Page	39
19. Figure 5.3.7 – Admin Dashboard	39
20. Figure 5.3.8 – Add Question Page	40
21. Figure 5.3.9 – SuperAdmin Page	40
22. Figure 5.3.10 – Manage Students / Manage Faculty Page	41
23. Figure 5.3.11 – Voice Training Page	41
24. Figure 5.3.12 – Manage User Page	42
25. Figure 5.3.13 – Manage Content	42
26. Figure 5.3.14 – Create Faculty	43

Chapter 1

Introduction

Today's campus placement processes are highly competitive and complex, posing many challenges for students. Often students face inadequate information on job requirements and lack structured practice opportunities. For example, one industry report notes that without modern support systems, students have “inadequate information on skill requirements and job roles, hindering effective preparation” and suffer from a “lack of mock tests and interview practice sessions”. In practice this leads to low confidence and insufficient preparedness. Furthermore, universities frequently fail to provide a centralized resource portal for placement preparation. This can result in inconsistent access to study materials, company profiles, and past interview experiences and overall inefficiency. In fact, educators note that many students “often lack the necessary preparation for placement interviews, group discussions, or aptitude tests,” directly impacting placement outcomes. These challenges – from fragmented information to limited self-assessment tools – highlight the need for a systematic, technology-driven preparation platform.

1.1 Motivation and Overview

To address these gaps, we propose a Placement Preparation System SYNRX (Speak Your Next Round Experience) – a unified web platform for campus-placement preparation. The motivation is to improve student readiness by providing centralized learning resources, adaptive practice modules, and analytics. Recent advances in educational technology show that data-driven platforms can greatly enhance learning: analytics make education “very personalized, adaptive, and engaging, able to identify better learning outcomes”. By applying these principles, the Placement Preparation System SYNRX aims to guide each student through tailored study paths and feedback. In addition, building such a system taps into current trends: many students and institutions are investing in online learning ecosystems and “placement portals” to streamline preparation and recruitment. Many students previously faced a fragmented

preparation process, relying on multiple platforms with inconsistent content and limited guidance. This reinforces the need for a unified system that brings practice tests, tutorials, and performance tracking together in one platform.

1.2 Objectives

The main objectives of the Placement Preparation System SYNRR are:

- **Practice Modules:** Provide dedicated practice tests and study materials in key areas (aptitude, technical, HR/interview questions) so students can actively prepare.
- **Question Management:** Maintain a rich question bank (aptitude problems, coding challenges, HR scenarios) that students can use in mock tests.
- **Performance Tracking:** Track each student's performance on practice tests (scores, time, strengths/weaknesses) and present analytics (e.g. accuracy charts, progress reports) to help them improve.
- **Admin Controls:** Offer an admin interface where faculty or placement officers can add/edit questions, manage tests, and view aggregate analytics of student performance.
- **User Management and Security:** Allow users to register securely and manage their profiles, ensuring only authorized access to content.

These features collectively aim to create a focused training environment. In particular, adding systematic performance feedback addresses the “absence of systematic assessment tools to evaluate students’ readiness” noted as a pain point. By achieving these objectives, our system will help bridge the gap between student preparation and industry expectations.

1.3 Summary of Similar Systems

Several existing platforms offer placement-prep resources, but they have limitations. PrepInsta is a popular subscription-based service that provides coding and aptitude courses. It has grown into a broad “career readiness ecosystem” with AI-driven personalization. GeeksforGeeks (GfG) offers free tutorials and question banks on quantitative aptitude, programming, and interview experiences. However, both have

drawbacks in a campus context. Many of their offerings are static content: students must manually navigate articles and mock tests without integrated tracking. For example, GfG’s placement section lists numerous topics and practice problems, but it does not adapt to individual student progress or provide consolidated analytics. As a result, colleges often still lack control over the process. Moreover, most such platforms require separate accounts or subscriptions, limiting institutional integration. In practice, the absence of a single centralized portal means inefficiencies persist – one recent survey argues that lacking a unified placement platform “can lead to inefficiencies, miscommunication, and missed opportunities”. Thus, while existing systems are helpful starting points, our Placement Preps System differentiates itself by offering college-level admin oversight, consolidated learning paths, and data-driven feedback in one place.

1.4 Report Organization

The remainder of this report is organized as follows.

Chapter 2

Software Requirement Analysis

Requirement analysis is a crucial phase that identifies what the Placement Preparation System SYNRX should achieve and how it should operate. It focuses on understanding student needs, analyzing existing preparation gaps, and ensuring the final system aligns with placement processes. The gathered requirements help define system boundaries, functionalities, user expectations, performance goals, and constraints.

2.1 Requirement Analysis

Requirement analysis identifies what the system must do and how it should perform. It includes:

2.1.1 Functional Requirements

1. **User Registration & Login:** Students and admins can create accounts and securely log in.
2. **Practice Test Module:** Students can select and take practice tests in areas like Aptitude, Coding, and HR interviews.
3. **Question Bank Management:** Admins can add, edit, or delete questions for each category (aptitude, technical, HR).
4. **Test Execution & Scoring:** The system presents questions to students, collects answers, automatically evaluates results, and stores scores.
5. **Performance Dashboard:** After each test, students see detailed results (score, correct answers, time taken) and can view historical performance charts.
6. **User Profile:** Students can view and update their personal information (e.g. name, email).
7. **Admin Controls:** The admin interface allows viewing user lists, monitoring test usage, and accessing overall analytics (average scores, question difficulty statistics, etc).

-
8. **Notifications/Feedback:** The system can notify users of new tests or results (e.g. via email) and provide contextual tips.

2.1.2 Non-Functional Requirements

1. **Performance:** Web pages and test modules should load quickly (e.g. pages load within 3 seconds) and test evaluations should be near-instant.
2. **Scalability:** The system should support a large number of concurrent users without degradation, using scalable server or cloud infrastructure.
3. **Accuracy:** Calculations (scoring, statistics) must be correct. Data integrity and precise result generation are essential.
4. **Security:** User credentials and data must be protected (encrypted passwords, secure sessions). The system should prevent unauthorized access.
5. **Usability:** Interfaces must be intuitive and responsive, usable on both desktop and mobile devices. The design should be clean and consistent for ease of use.
6. **Reliability:** The system should handle errors gracefully and maintain high availability during placement seasons.
7. **Maintainability:** Admins should be able to update content (questions/tests) without modifying code. The codebase should be well-organized for future updates.

These requirements ensure a robust, user-friendly preparation system that can grow with the institution's needs.

2.2 Feasibility Analysis

The feasibility study evaluates whether the project is practical and viable.

2.2.1 Technical Feasibility

The required technologies—such as web frontend frameworks, server-side JavaScript runtime environments, and NoSQL databases—are well-established, open-source tools. Technologies like HTML, CSS, and JavaScript can be used for the user interface; Node.js for server-side development; and MongoDB for the database layer. Additionally, there are ample libraries available for implementing features such as user

authentication, data visualization, and analytics. The expected scale of the placement preparation system is well within the capabilities of standard web infrastructure, making the project technically feasible.

2.2.2 Economic Feasibility

Since free/open-source tools can be used, development costs are low. Hosting costs are minimal (simple server or cloud). No specialized hardware is needed beyond a basic server and student devices. This makes the project economically viable for academic institutions.

2.2.3 Operational Feasibility

The system aligns with the operational needs of students, administrators, and placement officers. It should be easy for non-technical users to operate: students only need a browser to take tests, and admins have a simple web interface for content management. Training requirements are minimal. The system directly addresses the operational pain points in placement preparation.

2.2.4 Time Feasibility

Given a typical one-semester or one-year project timeline, the system's scope is reasonable. Core features (registration, test-taking, analytics) can be developed incrementally. Existing libraries and frameworks speed up development. There is sufficient time to design, implement, test, and deploy a functional prototype before the placement season.

Overall, the Placement Preps System is feasible with current technologies, resources, and timelines.

2.3 Module Description

The system is divided into three main functional modules: Student Module and Admin Module, SuperAdmin Module. Each module encapsulates related features and workflows, making the system easier to develop and maintain.

2.3.1 Student Module

The Student Module is the interface for campus candidates preparing for placements. It provides the following functionalities:

1. **Registration/Login:** Students sign up with basic details (name, email, password) and log in. The system verifies credentials and redirects them to their dashboard.
2. **Dashboard:** After login, students see their practice options and performance summary. The dashboard displays available tests (by category) and overall progress charts (e.g. average scores, best/worst subjects).
3. **Take Practice Tests:** Students select an Aptitude, Technical, or HR test. The system presents questions one by one or all at once. Students submit answers within a time limit.
4. **View Results:** Upon submission, the system scores the test and shows results immediately. This includes the student's score, correct answers, explanations (if any), and how they rank (e.g. percentile).
5. **Performance History:** Students can review past test results. The system generates analytics (graphs of scores over time, strengths/weaknesses per topic). This feedback loop helps students focus their study.
6. **Profile Management:** Students can update their profile details and reset their password.

The module's goal is to make preparation engaging and trackable. It hides complexity (all grading and statistics are computed in the background) so that students can focus on learning.

2.3.2 Admin Module

The Admin Module is for faculty or placement officers who manage the content and monitor student progress. Its functions include:

1. **Admin Login:** Admins authenticate with higher-privilege credentials.
2. **Question Bank Management:** Admins can create new questions or tests for each category. They can set question text, answer choices, correct answers, and tags (e.g. topic).

-
3. **Test Configuration:** Admins assemble practice tests from the question bank. They can set time limits, number of questions per test, and whether explanations are shown.
 4. **User Management:** Admins can view the list of registered students, deactivate accounts, or assign users to groups (e.g. batch, department).
 5. **Analytics & Reports:** The system collects data on all student test attempts. The Admin Dashboard presents summary reports: average scores per test, question difficulty analysis (e.g. percentage of students answering each question correctly), and user engagement statistics. Graphs and tables help administrators identify weak topics across the cohort.
 6. **System Logs:** Admins can review system logs or audit trails (e.g. login history, changes to questions) for security and maintenance.

By centralizing these controls, the Admin Module simplifies the placement cell's work. It ensures the system content stays up-to-date and gives visibility into student outcomes.

2.3.3 SuperAdmin Module

The SuperAdmin Module is the highest-privilege module in the Placement Preps System. It provides complete control over platform-wide operations, system configuration, and user-role management. While the Admin Module focuses on day-to-day test and student management, the SuperAdmin Module oversees the entire ecosystem, ensuring smooth functioning, security, and organizational control.

The responsibilities of the SuperAdmin include managing all admin accounts, monitoring system health, handling platform-level configurations, and ensuring that content and users adhere to institutional policies.

1. **SuperAdmin Authentication:** SuperAdmin log in using special high-security credentials. Additional security layers may be applied. Upon login, the system grants access to all administrative controls and dashboards.
2. **Admin Account Management:** Create new Admin accounts for departments, faculties, or placement officers. Edit, suspend, activate, or delete Admin accounts. Assign specific permissions or access levels to different Admins. Reset Admin passwords or enforce password policies for security.

-
3. **System-Wide Monitoring:** View activities of all Admins and Students across the platform. Monitor usage trends, such as: Total test attempts, Number of new users, Category-wise activity (Aptitude, Technical, HR). Detect unusual activities, unauthorized access, or potential system misuse.
 4. **Global Content Oversight:** Oversee all question banks created by Admins. Approve or reject newly uploaded question sets (optional workflow). Delete duplicate, outdated, or incorrect questions from the platform. Manage system-wide test templates that all Admins can use
 5. **Data & System Configuration:** Configure platform settings such as: Default test duration, Difficulty distribution, allowed categories, Email/notification settings, Manage storage locations for datasets and backups. Configure API or integration settings if required.
 6. **Advanced Analytics & Reports:** Access fully consolidated analytics including: Institution-wide overall performance, Admin performance metrics (e.g., content contributions), Student performance across all batches and departments, Trending skills, weak areas, and high-demand topics, Export reports in formats such as PDF, Excel, or CSV.
 7. **Database Management & Backup Controls:** Trigger or schedule automatic database backups. Restore data from backup points (in emergencies). Manage archived logs, datasets, and performance history.
 8. **Security and Audit Controls:** View complete audit trails of all user activities.
 - Track system logs such as:
 - Login attempts
 - Question modifications
 - Test configuration changes
 - Admin permission updates
 - Detect and prevent suspicious or inconsistent behaviour.

The SuperAdmin Module acts as the backbone of system governance. It ensures: High-level security, Smooth system functioning, Proper control over Admins, Reliable platform-wide operations. And by controlling global settings, overseeing content quality, and managing all administrative activities, the SuperAdmin Module makes the Placement Preps System scalable, secure, and institution-ready.

2.4 Use Case Descriptions

The Placement Preparation App supports three types of actors: Student, Admin, and SuperAdmin. Each actor interacts with the system through specific use cases as shown in the diagram. The following section provides detailed descriptions for each use case.

2.4.1 Student Use Cases

Use Case 1: Sign Up / Login

- **Actor:** Student
- **Description:** The student creates an account using email and password or logs into an existing account.
- **Outcome:** Student is authenticated and redirected to the dashboard.

Use Case 2: Attempt Quiz

- **Actor:** Student
- **Description:** The student selects a quiz from Aptitude, Technical, or HR categories and attempts it within a time limit.
- **Outcome:** Answers are submitted and stored for evaluation.

Use Case 3: Browse Topics

- **Actor:** Student
- **Description:** Allows students to explore various subjects, chapters, or categorized learning topics.
- **Outcome:** Students gain structured access to study content and explanations.

Use Case 4: Access Study Materials

- **Actor:** Student
- **Description:** The student can view or download notes, PDFs, tutorials, video links, and other academic content uploaded by faculty or admin.
- **Outcome:** Student improves preparation through available study material.

Use Case 5: View Job Alerts

- **Actor:** Student

-
- **Description:** Students can see job openings, internship notifications, placement drives, deadlines, and company descriptions.
 - **Outcome:** Student stays updated with employment opportunities.

Use Case 6: View Results & Analytics

- **Actor:** Student
- **Description:** The system displays quiz results, correct answers, scores, performance charts, and topic-wise strengths/weaknesses.
- **Outcome:** Students track progress and identify improvement areas.

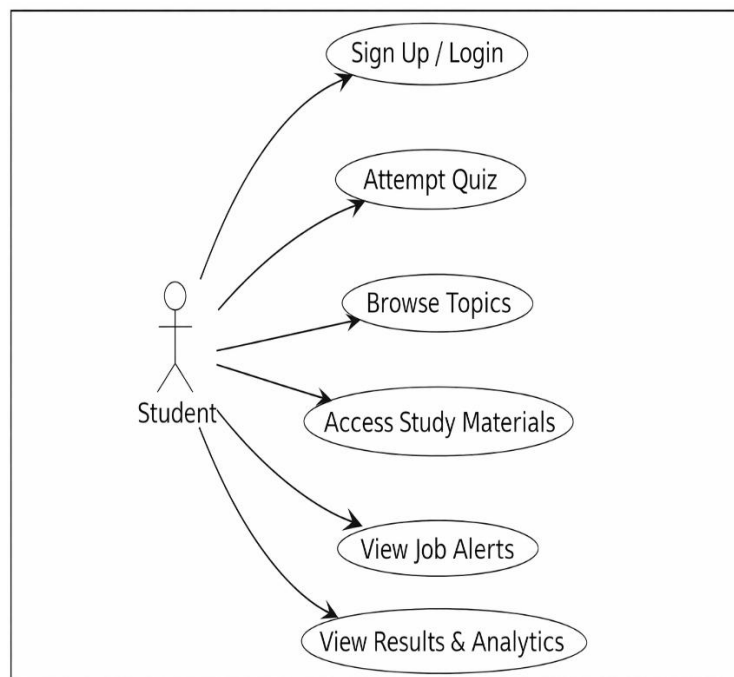


Figure 2.4.1 Use Case Diagram of User

2.4.2 Admin Use Cases

Use Case 1: Login

- **Actor:** Admin
- **Description:** Admin members authenticate using valid credentials.
- **Outcome:** Access granted to Admin dashboard and tools.

Use Case 2: Upload Study Material

- **Actor:** Admin

-
- **Description:** Admin can upload PDFs, images, notes, assignments, or links for student learning.
 - **Outcome:** Materials become accessible to all students in the system.

Use Case 3: Create / Manage Quizzes

- **Actor:** Admin
- **Description:** Admin can create new quizzes by adding questions, editing existing quizzes, organizing categories, or scheduling tests.
- **Outcome:** Updated quizzes become available for students to attempt.

Use Case 4: Evaluate Results

- **Actor:** Admin
- **Description:** Admin can view student submissions, check difficulty patterns, and validate automated evaluations.
- **Outcome:** Admin monitors student performance and improves question quality.

Use Case 5: Send Notifications

- **Actor:** Admin
- **Description:** Admin sends announcements about new quizzes, study materials, deadlines, or guidance messages.
- **Outcome:** Students get timely updates.

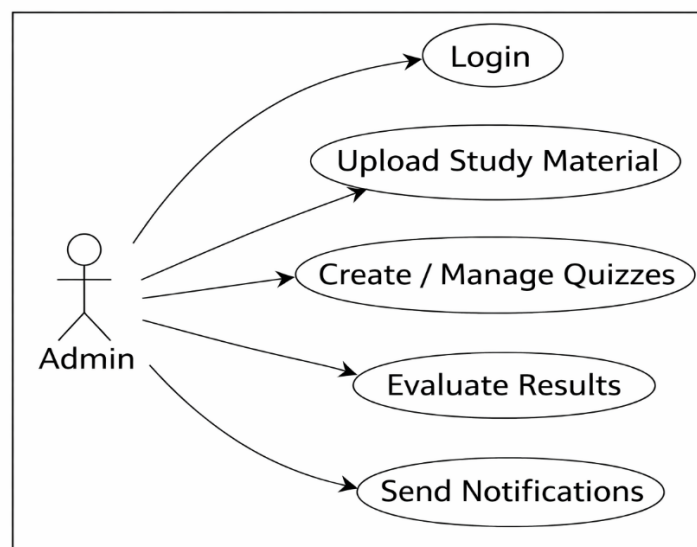


Figure 2.4.2 Use Case Diagram of Admin

2.4.3 SuperAdmin Use Cases

Use Case 1: System Maintenance

- **Actor:** SuperAdmin
- **Description:** SuperAdmin oversees system performance, manages server configurations, backups, settings, and ensures smooth operation.
- **Outcome:** System remains functional, secure, and up to date.

Use Case 2: View Overall Reports

- **Actor:** SuperAdmin
- **Description:** SuperAdmin can view global-level analytics including number of students, usage trends, quiz performance, and activity summaries.
- **Outcome:** SuperAdmin gains insights into system-wide behaviour.

Use Case 3: Manage Students

- **Actor:** SuperAdmin
- **Description:** SuperAdmin can create, edit, activate, or deactivate student accounts.
- **Outcome:** Student database is fully under administrative control.

Use Case 4: Manage Faculty Accounts

- **Actor:** SuperAdmin
- **Description:** SuperAdmin can add new faculty members, update details, assign roles, or remove them as needed.
- **Outcome:** Proper hierarchical control and faculty management is maintained.

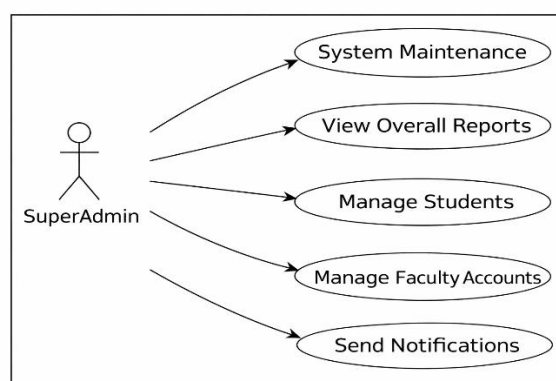


Figure 2.4.3 Use Case Diagram of SuperAdmin

2.5 System Architecture

The system follows a **three-tier architecture**, separating the user interface, application logic, and data storage. The key components are:

- **Frontend (Presentation Layer):** Web-based user interfaces for both students and admins. Built with HTML/CSS/JavaScript (or a framework like React), it provides responsive screens for registration, test-taking, dashboards, etc. This layer handles input/output (forms, buttons) and sends requests to the backend.
- **Backend (Application Logic Layer):** The core server application (e.g. implemented in Python, Node.js) that contains the business logic. It handles user authentication, test generation, scoring algorithms, analytics computations, and enforces rules (time limits, access rights). Controllers in this layer process incoming requests from the frontend and interact with the database.
- **Database (Data Management Layer):** A NoSQL database (like MongoDB) that stores all persistent data: user accounts, questions, test definitions, and test results. The database schema is designed to support the ER model described in Chapter 3. Queries retrieve and update data as needed.

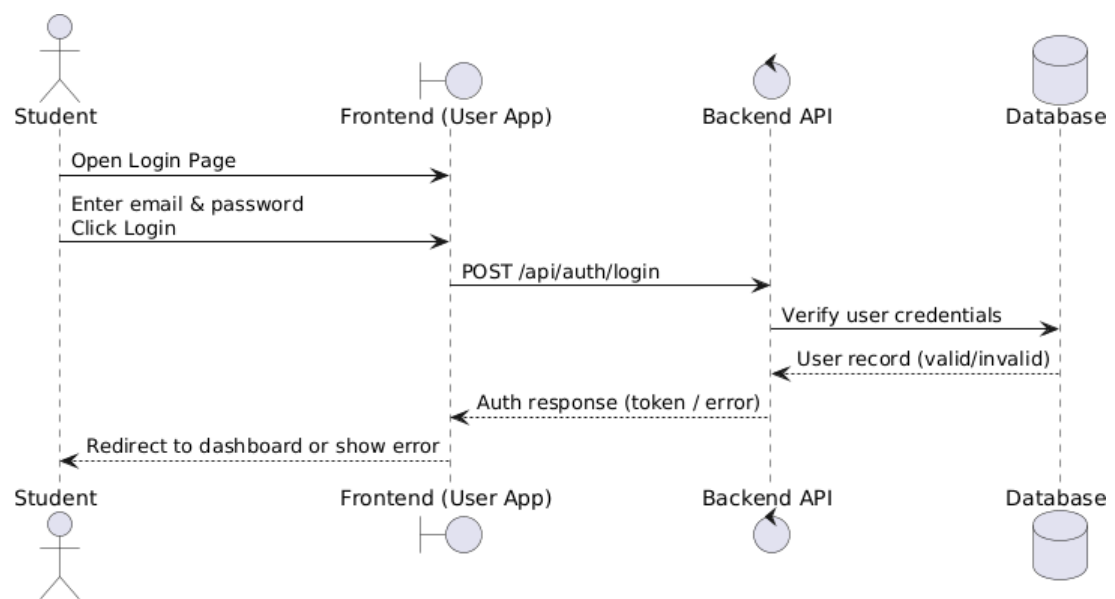


Figure 2.5 Sequence Diagram for Login

Chapter 3

System Design

The System Design translates the requirements into a concrete architecture and models. This chapter details the system's architecture, data flow diagrams, UML diagrams, and the ER diagram of the database.

3.1 Data Flow Diagrams

Data Flow Diagrams (DFDs) illustrate how data moves through the system. We provide diagrams at multiple levels of detail.

3.1.1 Level 0 (Context Diagram):

The highest-level DFD shows the system as a single process with external entities. In our case, the main external entities are *Student* and *Admin*. The Student sends requests (e.g. login, take test, view results) into the system, and the system returns responses (test questions, scores, reports). The Admin sends administrative commands (add questions, view reports) and receives confirmation and analytics data. The Level-0 DFD abstracts away internal processing, focusing on inputs and outputs (Figure below).

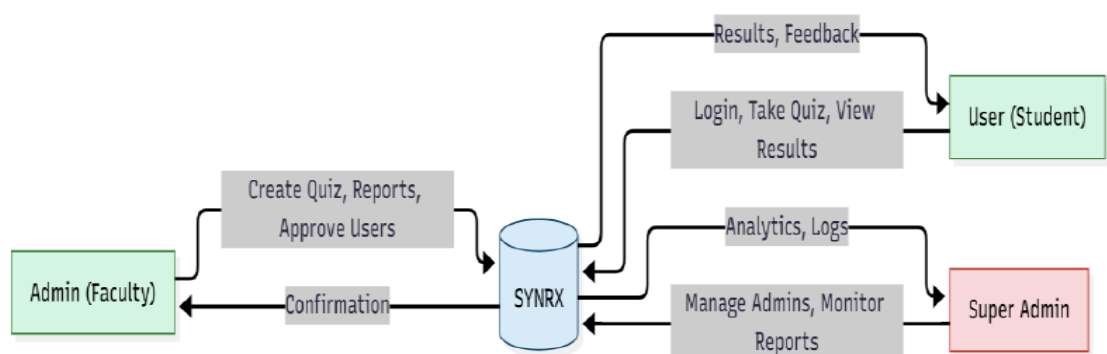


Figure 3.1.1 Level 0 Data Flow Diagram

3.1.2 Level 1 DFD:

The system is divided into several major subprocesses that work together to support the online examination workflow. The primary processes include User Authentication, Test Management, and Result Processing. When a student attempts to log in, the Authentication process validates the submitted credentials by comparing them with stored records in the database. Once authenticated, the student is allowed to access examination modules.

During an exam session, the Test Management process interacts with the database to fetch the set of questions mapped to the selected test. These questions are then presented to the student through the Test Interface. As the student submits answers, the Scoring and Evaluation subprocess compares the responses against correct answers stored in the database. After the evaluation is completed, the Result Processing process stores the generated results in the database. These results can later be retrieved for report generation, performance review, or administrative analysis.

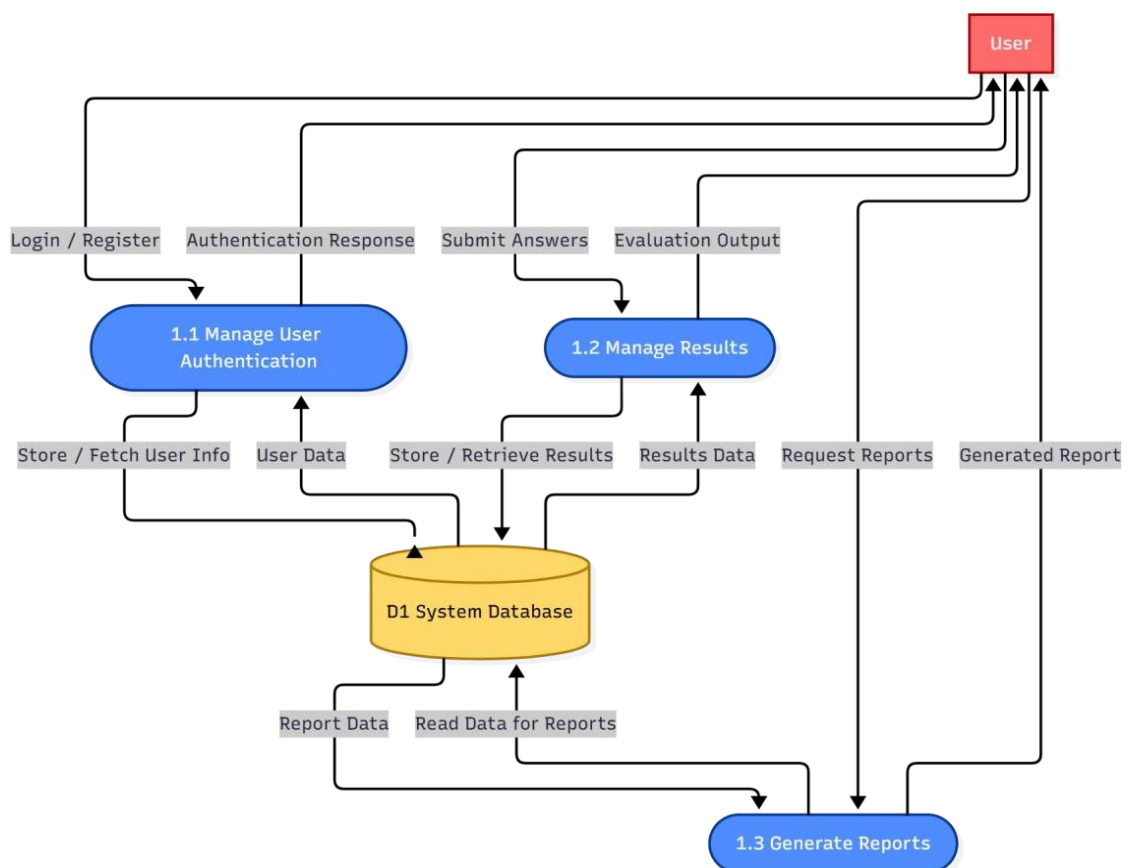


Figure 3.1.2 Level 1 Data Flow Diagram

3.1.3 Level 2 DFD:

Further decomposition details individual tasks. For instance, the User Authentication process can be split into “Verify Credentials” and “Load Profile”. The Test Engine can be split into “Select Questions” and “Present Questions”. These lower-level DFDs ensure every data flow is accounted for. (For brevity, we describe them conceptually; full diagrams would be included in an appendix.) Overall, the multi-level DFDs clarify the internal workings, showing exactly how student inputs traverse the system and what outputs result.

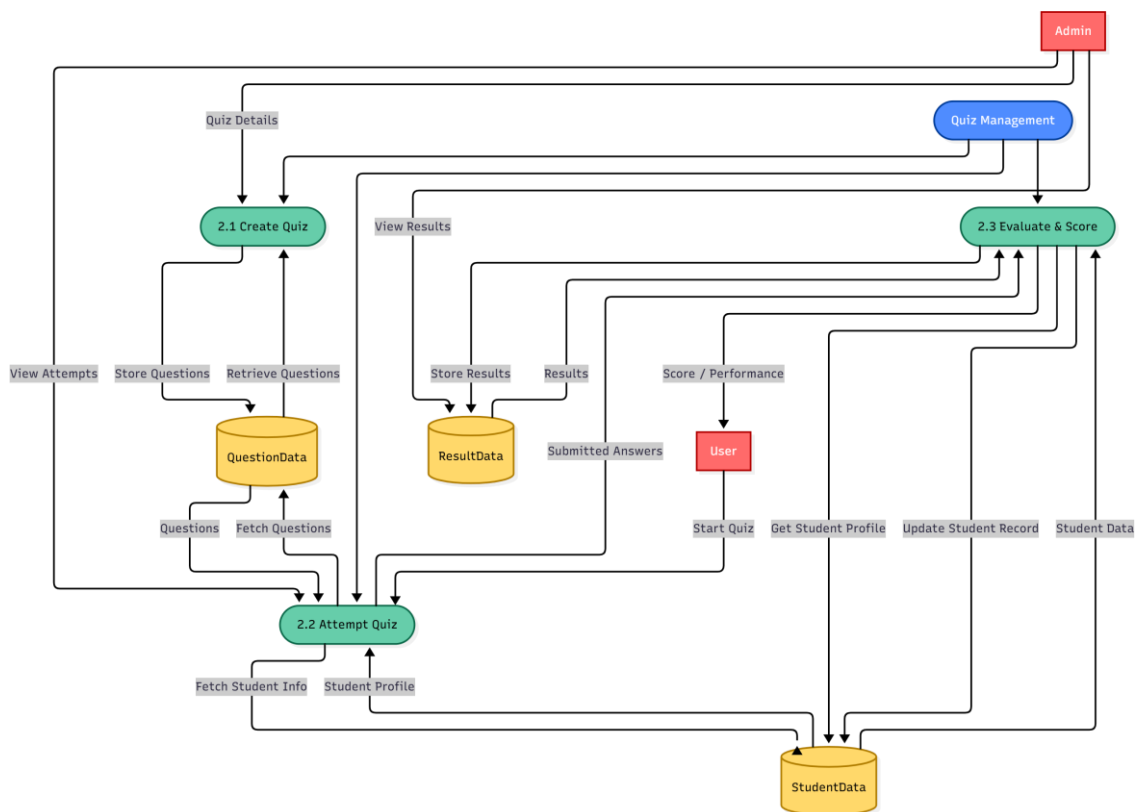


Figure 3.1.3 Level 2 Data Flow Diagram

3.2 UML Diagrams

We use UML (Unified Modelling Language) diagrams to model the system’s structure and behaviour:

3.2.1 Class Diagram

The UML Class Diagram represents the core classes in the system along with their attributes, operations, and interrelationships. Key classes typically include Student, Admin, Test, Question, and Result (or Performance Report in our system).

1. **Student** contains attributes such as *studentID*, *name*, and *email*. It provides operations like *takeTest()* and *viewReport()*.
2. **Question** includes *questionID*, *text*, *options*, and *correctAnswer*, along with the operation *checkAnswer()*.
3. **Test** aggregates a set of *Question* objects and maintains details such as *testID* and *testDate*.
4. **Admin**, a subclass of *User*, can add questions and view analytics using operations such as *addQuestion()* and *viewAnalytics()*.
5. The **Result** (or *PerformanceReport*) class links a *Student* to a *Test* and stores *score*, *timeTaken*, and report-generation methods.

Associations illustrate relationships:

- A *Student* takes one or more *Tests*.
- A *Test* contains multiple *Questions*.
- A *PerformanceReport* is generated after a student completes a test.

This class diagram ensures the database schema and backend architecture follow the intended functional structure of the system.

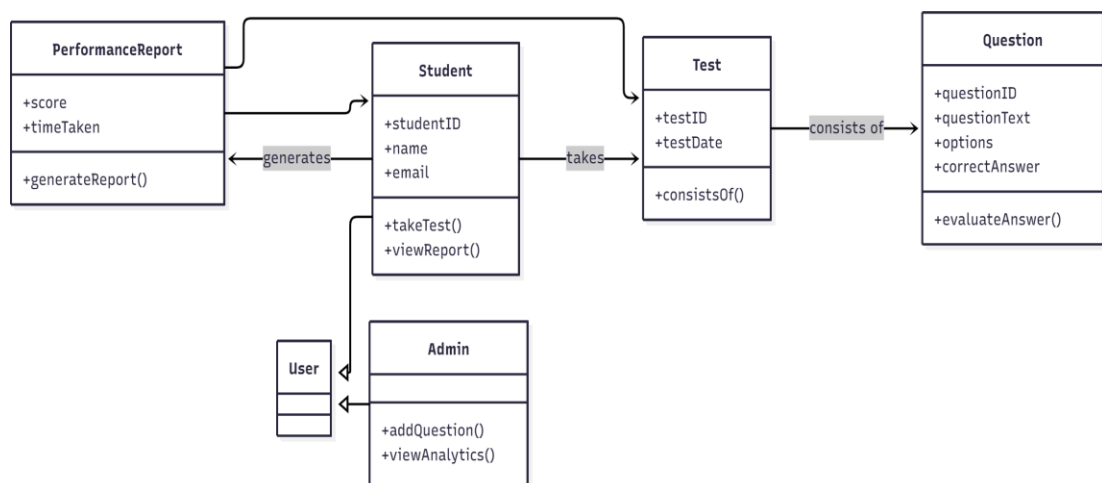


Figure 3.2.1 Class Diagram

3.2.2 Sequence Diagram

The sequence diagram illustrates the complete workflow of how a student interacts with the system while attempting a quiz. It shows communication between four main components: Student, Frontend Application, Backend API, and the Database. The flow is as follows:

1. **Student Opens the Quiz Page:** The interaction begins when the student navigates to the quiz page in the application. The Frontend (User App) receives this request.
2. **Frontend Requests Quiz Data:** The Frontend sends a **GET** request to the Backend API (GET /api/quiz/:id) to retrieve the quiz details and associated questions.
3. **Backend Fetches Quiz Information:** The Backend queries the **Database** to fetch the quiz metadata and its related questions. The Database returns the complete quiz data to the Backend.
4. **Backend Sends Quiz Data to Frontend:** The Backend forwards the quiz dataset back to the Frontend, which then renders the quiz for the student.
5. **Student Submits Answers:** After answering the questions, the student submits the quiz. The Frontend sends the answers to the Backend via a **POST** request (POST /api/quiz/:id/attempt).
6. **Backend Saves the Attempt and Computes the Score:** The Backend stores the student's attempt in the Database, computes the score, and prepares the analytics.
7. **Database Returns Confirmation and Score:** The Database responds with confirmation that the attempt was saved successfully, along with the calculated score and related data.
8. **Backend Returns Results to Frontend:** The Backend forwards the score and analytics to the Frontend.
9. **Frontend Displays Results to Student:** Finally, the Frontend presents the score, performance summary, and analytics to the student.

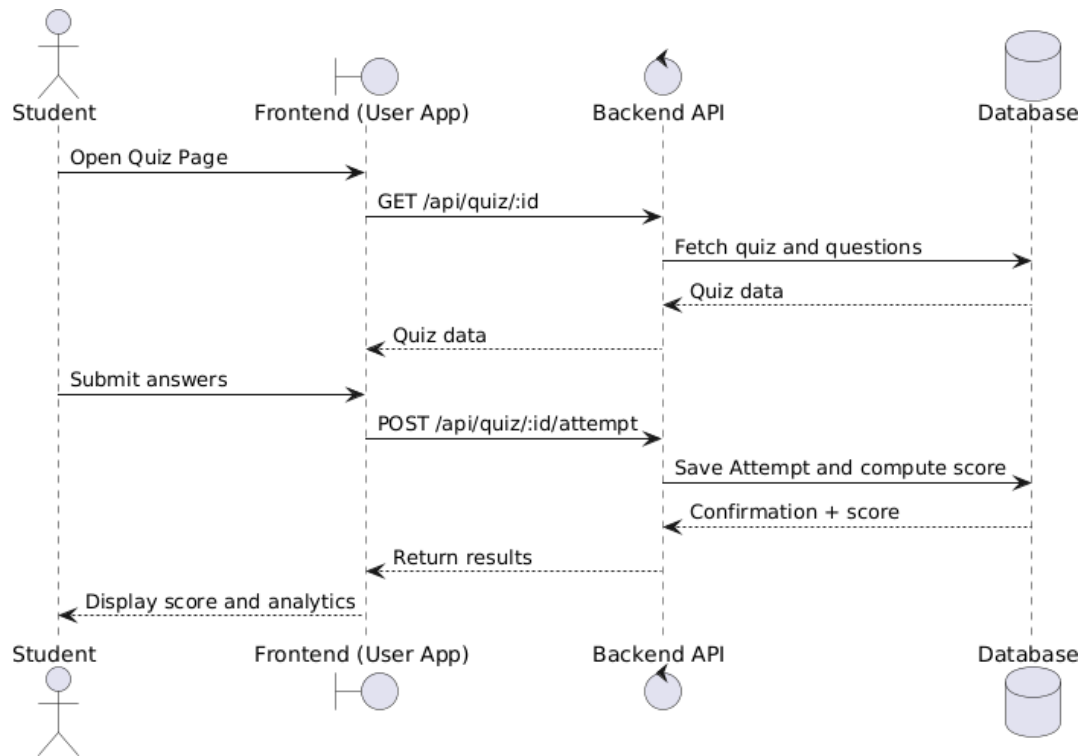


Figure 3.2.2 Sequence Diagram

3.2.3 Object Diagram

An Object Diagram is a snapshot of system instances at a particular point in time.

For instance, an object diagram may show:

- A *Student* object: studentName = "Yash"
- A *Test* object: testName = "AptitudeTest1"
- Multiple *Question* objects with actual values answered by the student

This helps verify whether the class design can model real-world scenarios, such as completed tests and stored performance reports.

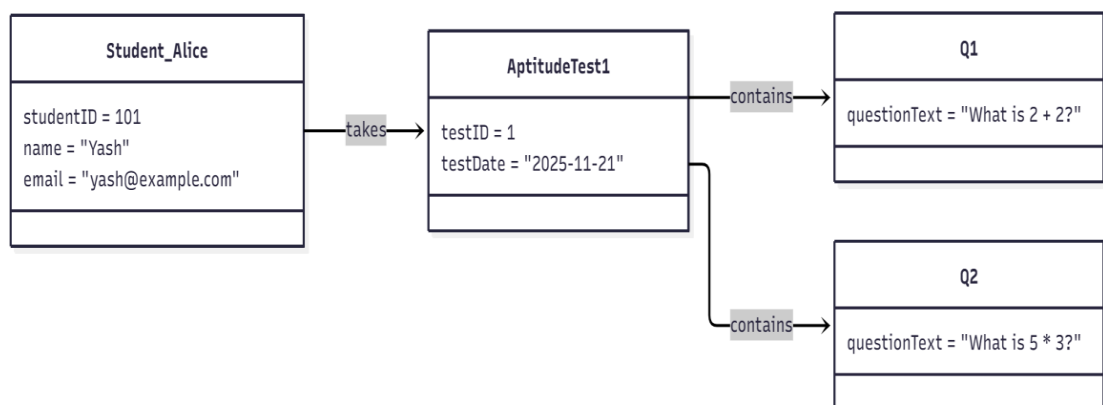


Figure 3.2.3 Object Diagram

3.2.4 Collaboration Diagram

A Collaboration Diagram (also called a Communication Diagram) emphasizes object relationships and the order of interactions.

In a scenario such as “**Admin Adds a Question**”, the collaboration diagram might illustrate:

- A *UI* object interacting with a *Question Manager*
- A *Question Manager* interacting with the *Database*
- Numbered messages like:
 1. Login ()
 2. addQuestion ()
 3. saveToDB ()

Compared to the sequence diagram, the collaboration diagram focuses more on **roles**, **links**, and **object responsibilities**. Using both diagrams for key actions (student taking test, admin adding questions) ensures a well-defined and robust system design.

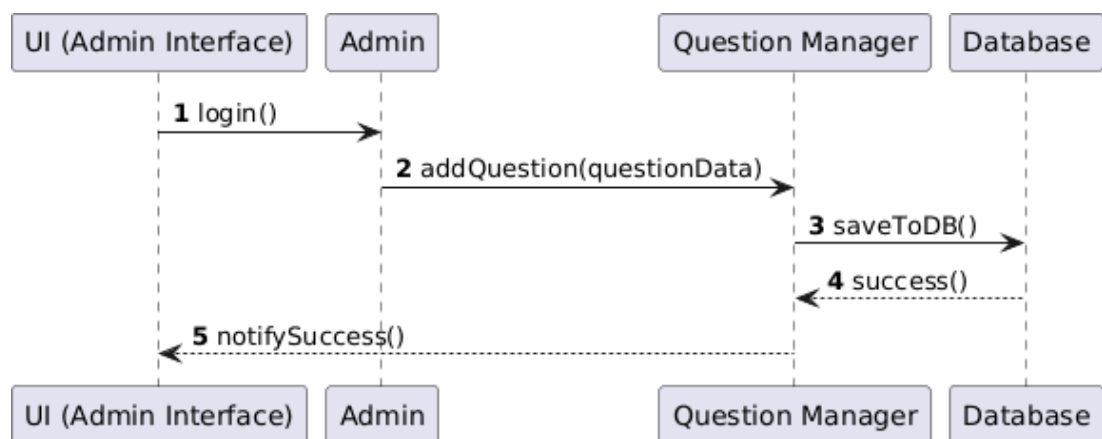


Figure 3.2.4 Collaboration Diagram

3.3 ER Diagram

The Entity–Relationship (ER) diagram represents the logical data structure of the Placement Preparation System. It identifies the core entities, their attributes, and the relationships between them. The ER model ensures that the database design supports all system functionalities such as user management, test creation, question storage, and performance evaluation.

Major Entities

1. SuperAdmin

The SuperAdmin oversees the entire system and manages both Admins and Students.

Attributes: SuperAdmin (PK), name, email, password

2. Admin

Admins are responsible for creating and managing Tests and Questions.

Attributes: adminID (PK), name, email, password

3. Student

Students participate in tests and generate performance records.

Attributes: studentID (PK), name, email, password

4. Test

Each Test is created by an Admin and contains multiple Questions.

Attributes: testID (PK), title, category, totalQuestions, duration

5. Question

A Test contains multiple Questions. Each Question stores options and the correct answer.

Attributes: questionID (PK), questionText, options, correctOption, category

6. Performance

This entity records the result of a Student taking a specific Test.

Attributes: performanceID (PK), studentID (FK), testID (FK), score, dateTaken, timeTaken

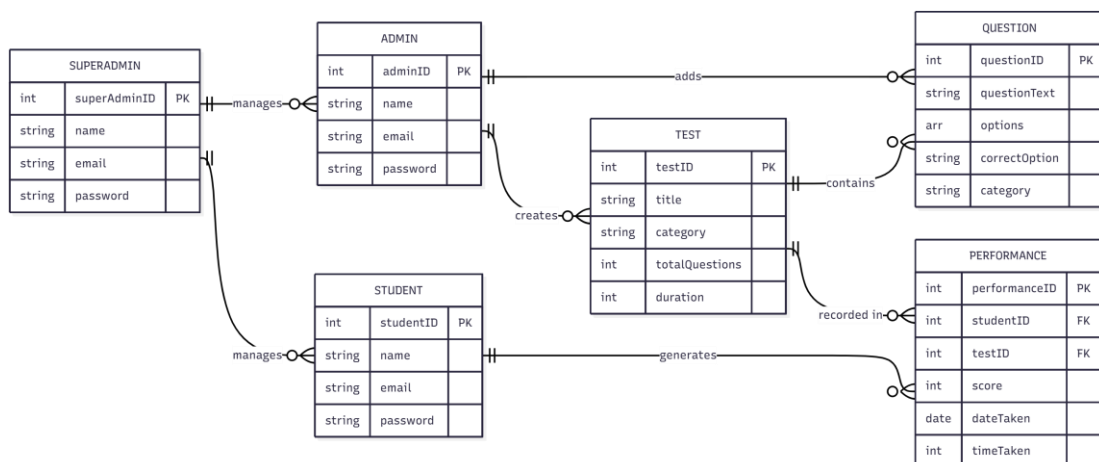


Figure 3.3 Er Diagram

Relationships

- **SuperAdmin → Admin**
One SuperAdmin can manage many Admins.
- **SuperAdmin → Student**
One SuperAdmin can manage many Students.
- **Admin → Test**
An Admin can create many Tests.
- **Admin → Question**
An Admin can add many Questions.
- **Test → Question**
A Test contains multiple Questions.
(One-to-many relationship)
- **Student → Performance**
A Student can generate many Performance records.
(One-to-many)
- **Test → Performance**
A Test can have many Performance records for different students.
- **Performance acts as an associative entity** between Student and Test, representing the “Student takes Test” relationship and storing results such as score and timeTaken.

Purpose of the ER Model

This ER diagram ensures that all functional requirements of the system are covered:

- User management through SuperAdmin and Admin roles
- Test creation and Question storage
- Student test-taking and result tracking
- Data normalization to avoid redundancy
- Clear structure for implementing MongoDB/Mongoose collections

The ER model guides the actual database design and ensures consistency between backend implementation and system requirements.

Chapter 4

Implementation

Implementation is the phase where the system design is transformed into a working application. This chapter explains the technologies used, system modules, backend and frontend development processes, and how all components are integrated to form the final Placement Preparation System.

4.1 Technologies Used

The Placement Preparation System is developed using a combination of modern frontend and backend technologies. These tools ensure high performance, scalability, and a smooth user experience. The technologies are organized below into two main categories: Frontend Technologies and Backend Technologies.

4.1.1 Frontend Technologies

1. **HTML5**
Used for creating the structure and layout of the web pages.
2. **CSS3 / Tailwind CSS / Bootstrap**
Provides styling, responsiveness, and UI components for a clean and modern interface.
3. **JavaScript (ES6+)**
Adds dynamic functionality to the website and handles client-side logic.
4. **React.js** (*or your chosen framework*)
Used to develop modular, component-based front-end screens such as dashboards, quiz pages, and analytics.
5. **Axios / Fetch API**
Used for sending API requests from the frontend to the backend.
6. **Chart.js / Recharts / ApexCharts**
Enables graphical representation of performance analytics.

4.1.2 Backend Technologies

1. **Node.js with Express.js**

Used for building the server, APIs, routing, and business logic.

2. **MongoDB / MySQL**

Database used to store users, quizzes, questions, results, study materials, and analytics data.

3. **Mongoose MySQL Connector**

For modelling database schemas and interacting with the database.

4. **JWT (JSON Web Token)**

Handles secure authentication for Students, Admin, and SuperAdmin.

5. **bcrypt.js**

Encrypts passwords before storing them in the database.

6. **Cloud Storage / Multer**

Used for uploading study materials, images, or PDFs.

7. **RESTful API Architecture**

Enables communication between frontend and backend through structured routes.

8. **Nodemailer / Email Services**

Used for sending notifications or account-related emails (if implemented).

4.2 System Modules

The Placement Preparation System SYNRX is divided into three major functional modules, each designed to handle specific roles and responsibilities. These modules ensure smooth workflow, proper access control, and efficient management of system operations.

4.2.1 Student Module

The Student Module handles all core functionalities related to learners preparing for placements. It focuses on delivering study content, quizzes, performance tracking, and job-related resources.

Key Features:

1. **User Registration and Login:** Students create an account and authenticate securely.
2. **Browse Topics:** Students can view categorized topics to study.
3. **Access Study Materials:** Includes notes, PDFs, videos, and uploaded resources.
4. **Attempt Quizzes:** Students can take Aptitude, Technical, and HR quizzes.
5. **View Results & Analytics:** Displays detailed performance analysis and score progression.
6. **View Job Alerts:** Students can check upcoming opportunities and placement notifications.
7. **Manage Profile:** Update personal details, password, and preferences.

This module ensures an interactive and user-friendly environment for students to prepare effectively.

4.2.2 Admin Module

The Admin Module supports placement mentors or educators in managing quizzes, materials, and student evaluations.

Key Features:

1. **Secure Login:** Admin members authenticate through a dedicated portal.
2. **Upload Study Materials:** Upload notes, documents, and content for students.
3. **Create & Manage Quizzes:** Add questions, edit quizzes, and schedule tests.
4. **Evaluate Results:** Monitor quiz submissions and review student performance.
5. **View Student Submissions:** Detailed access to attempts and answer sheets (if implemented).
6. **Send Notifications:** Announcements related to quizzes, study materials, or placement updates.

Admin members ensure the content remains updated and aligned with placement requirements

4.2.3 SuperAdmin Module

The SuperAdmin Module provides the highest level of control over the system. It ensures that overall platform management, security, and data integrity are maintained.

Key Features:

1. **Manage Student Accounts:** Add, edit, activate, or deactivate students.
2. **Manage Faculty Accounts:** Create new faculty members, modify roles, or remove users.
3. **View Overall System Reports:** Global analytics across students, quizzes, and usage.
4. **Perform System Maintenance:** Server health checks, logs, and cleanup activities.
5. **Backup & Restore Data:** Ensures system data safety and recovery.
6. **Oversee Security & Logs:** Monitors system usage, login history, and suspicious activity.

This module ensures high-level administrative control over the entire platform.

4.3 Backend Implementation

The backend forms the core of the Placement Preparation System. It handles business logic, database communication, and user authentication.

4.3.1 REST API Development

1. APIs were created using **Node.js/Express** (or Python Flask/PHP based on your project).
2. Routes were categorized by modules:
 - /auth – login, registration

-
- /student – quizzes, materials, analytics
 - /faculty – quiz management, uploads
 - /superadmin – account management

Each API validates inputs, applies authentication, and returns JSON responses.

4.3.2 Database Implementation

1. User models (Student, Faculty, SuperAdmin) were stored with encrypted passwords.
2. Quiz collections/tables stored:
 - Question text
 - Options
 - Correct answer
 - Category (Aptitude/Technical/HR)
 - Quiz ID
3. Result tables stored:
 - Student ID
 - Quiz ID
 - Score
 - Timestamp

Queries were optimized using indexes and relationships.

4.3.3 Authentication & Security

- **JWT Token-based authentication** for secure login.
- Middleware to ensure access control:
 - Students cannot access faculty routes.
 - Faculty cannot access SuperAdmin routes.
- Password hashing using **bcrypt**.
- File uploads validated for size and format.

4.3.4 Business Logic

Backend handles:

- Quiz evaluation
- Score calculation
- Generating analytics data
- Filtering study materials
- Maintaining logs and system monitoring
- Sending notifications

4.4 Frontend Implementation

Frontend provides an interactive and user-friendly interface.

4.4.1 User Interface Development

Pages developed include:

- Home Page
- Login/Signup
- Student Dashboard
- Quiz Page
- Study Material Page
- Results & Analytics
- Faculty Dashboard
- SuperAdmin Dashboard

Each page is responsive on mobile and desktop.

4.4.2 Component-Based Design

If React was used:

- Components include:

-
- Navbar, Sidebar, QuizCard, ResultChart, ProfileForm, AdminTable
 - State management using:
 - React State
 - Context API
 - Redux (optional)

This modular design makes frontend easier to maintain.

4.4.3 Communication with Backend

- Axios/Fetch API used for sending requests.
- Token stored in: Cookies (secure)
- Error handling for login failures, missing fields, and server errors.

4.4.4 Displaying Analytics

- Graphs built using:
 - Chart.js
- Visualizations include:
 - Score progression
 - Topic-wise performance
 - Difficulty levels

4.5 Integration

Integration connects the frontend, backend, and database to work as a full system.

4.5.1 API Integration

- Frontend sends requests to backend through REST APIs.
- Backend processes the request and communicates with the database.
- Responses are returned as JSON for rendering UI.

4.5.2 Module Integration

- Login connects to role-based dashboards.
- Quiz system integrated with:
 - Question bank
 - Timer
 - Evaluation module
- Results integrated with:
 - Analytics Engine
 - Student Profile

4.5.3 Testing & Debugging

- Each API endpoint tested using Postman.
- UI tested across screen sizes.
- Errors logged and fixed at each layer.

4.5.4 Deployment

- System deployed using:
 - Render / Vercel / Netlify (Frontend)
 - Node server / Python server deployment
 - Database on MySQL/MongoDB Cloud

All parts were configured to work together securely and efficiently.

Chapter 5

Testing & Results

Testing is a crucial phase of software development used to evaluate the functionality, performance, reliability, and security of the system. This chapter explains the testing methods used, the detailed test cases, and the final results with screenshots.

5.1 Testing Methods

To ensure that the Placement Preparation System works as intended, multiple testing techniques were applied:

5.1.1 Unit Testing

Individual functions, API endpoints, and components were tested separately.

Examples:

- Student login API
- Quiz evaluation logic
- Faculty quiz creation functions
- SuperAdmin user management functions

This ensures that each function behaves correctly in isolation.

5.1.2 Integration Testing

This testing verified that combined modules work together smoothly.

Examples:

- Student quiz → evaluation → results generation
- Faculty creates quiz → student attempts quiz
- SuperAdmin adds faculty → faculty logs in

It ensures proper interaction between modules and databases.

5.1.3 Functional Testing

Performed to check whether the system meets the functional requirements.

Major features tested:

- Authentication
- Quiz flow
- Study material upload
- Analytics generation
- Notifications

Each feature was tested end-to-end.

5.1.4 User Interface (UI) Testing

Ensured that:

- Pages load correctly
- Buttons, forms, dropdowns function properly
- Layout is responsive on mobile/desktop

5.1.5 Performance Testing

Checked how the system performs under load:

- Multiple students attempting quizzes simultaneously
- Large question bank load time
- Dashboard analytics rendering

5.1.6 Security Testing

Tested for:

- Password encryption
- Unauthorized access attempts
- Role-based route restrictions (Student/Faculty/SuperAdmin)
- Token expiration and refresh

5.2 Test Cases

Below are sample test cases used during testing.

Test Case 1 – Login Test Case

Test Case ID	TC-01
Feature	User Login
Description	Verify Login with Valid Credentials
Input	Email & Password
Expected Output	User should be redirect to dashboard
Actual Output	As Expected,
Status	Passed

Figure 5.2.1 Login Test Case

Test Case 2 – Invalid Login Test Case

Test Case ID	TC-02
Module	Authentication
Test Scenario	Login with Incorrect Password
Precondition	User exists
Input	Wrong Password
Expected Output	Error Message “Invalid Credentials”

Figure 5.2.2 Invalid Login Test Case

Test Case 3 – Quiz Attempt Test Case

Test Case ID	TC-03
Module	Quiz Module
Test Scenario	Attempt Quiz and Submit
Precondition	Quiz must exit
Input	Selected answers
Expected Output	Score & Result generated
Actual Output	Correct results shown
Status	Pass

Figure 5.2.3 Quiz Attempt Test Case

Test Case 4 – View Analytics

Test Case ID	TC-04
Module	Analytics
Test Scenario	View Performance graph
Precondition	At least one quiz attempt
Input	Open analytics page
Expected Output	Charts load correctly
Actual Output	Working fine
Status	Pass

Figure 5.2.4 View Analytics Test Case

Test Case 5 – Faculty Create Quiz

Test Case ID	TC-05
Module	Quiz Management
Test Scenario	Create new quiz
Precondition	Faculty authenticated
Input	Question set & Quiz details
Expected Output	Quiz added to system
Actual Output	Added successfully
Status	Pass

Figure 5.2.5 Faculty Create Quiz Test Case

Test Case 6 – SuperAdmin Adds Faculty

Test Case ID	TC-06
Module	SuperAdmin Module
Test Scenario	Add new faculty account
Precondition	SuperAdmin login
Input	Faculty Email
Expected Output	Faculty mail sent and approve
Actual Output	Working fine
Status	Pass

Figure 5.2.6 SuperAdmin Add Faculty Test Case

Test Case 7 – Unauthorized Access Restriction

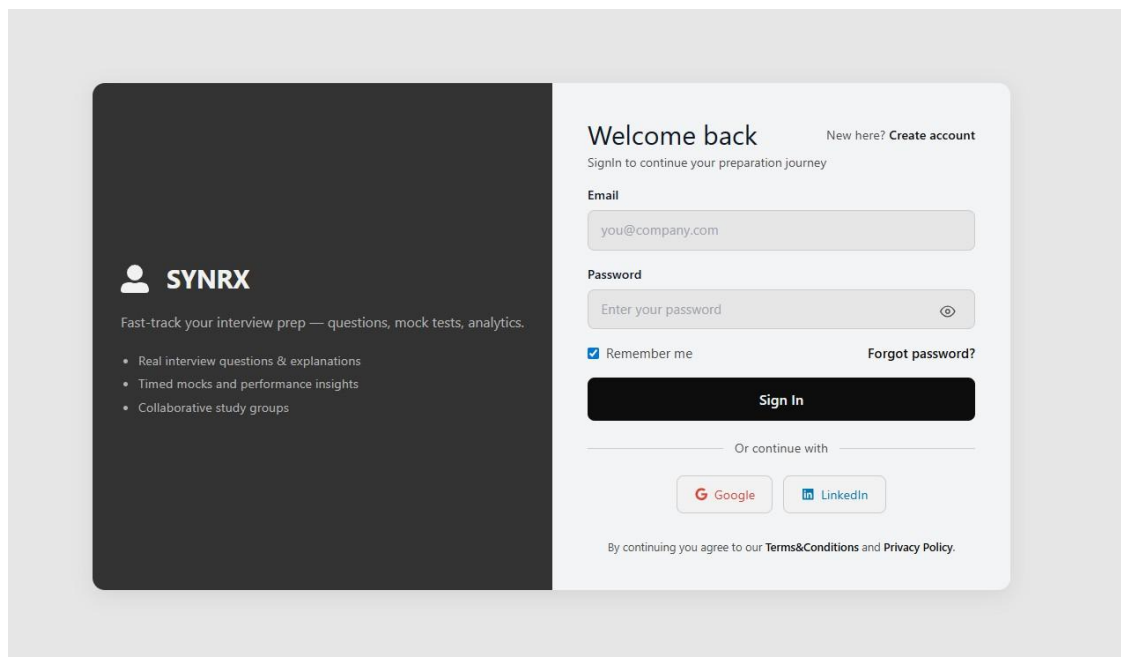
Test Case ID	TC-07
Module	Security
Test Scenario	Student trying to open admin route
Precondition	Student logged In
Input	Access Admin or SuperAdmin routes
Expected Output	“Unauthorized access”
Actual Output	Access correctly blocked
Status	Pass

Figure 5.2.7 Unauthorized Access Test Case

5.3 Results & Screenshots

Testing demonstrated that all functionalities work properly and the system meets the project requirements. The interface is responsive, quizzes run smoothly, analytics display correctly, and user roles behave as expected.

1. Login Page

*Figure 5.3.1 Login Page*

2. Student Dashboard

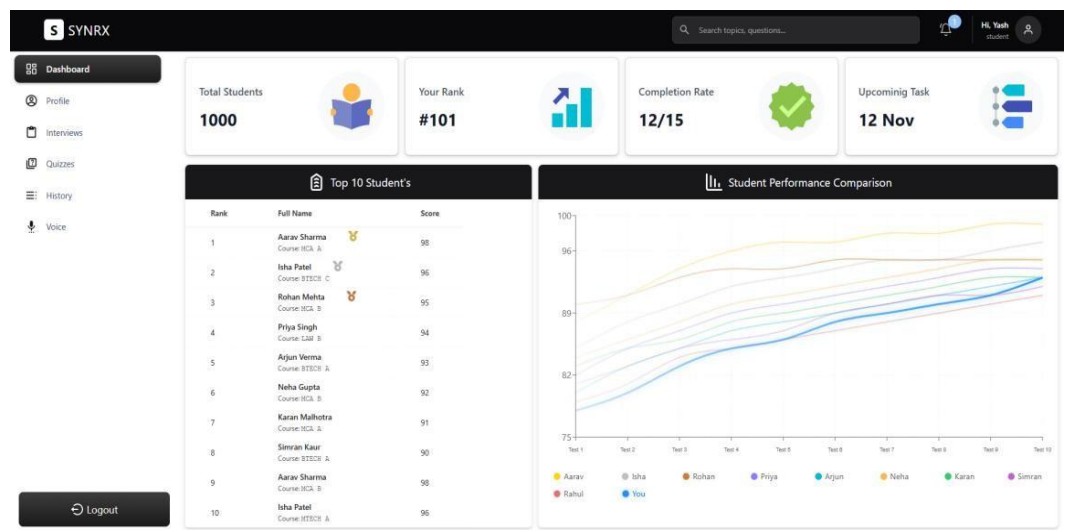


Figure 5.3.2 Student Dashboard Page

3. Quiz Listing Page

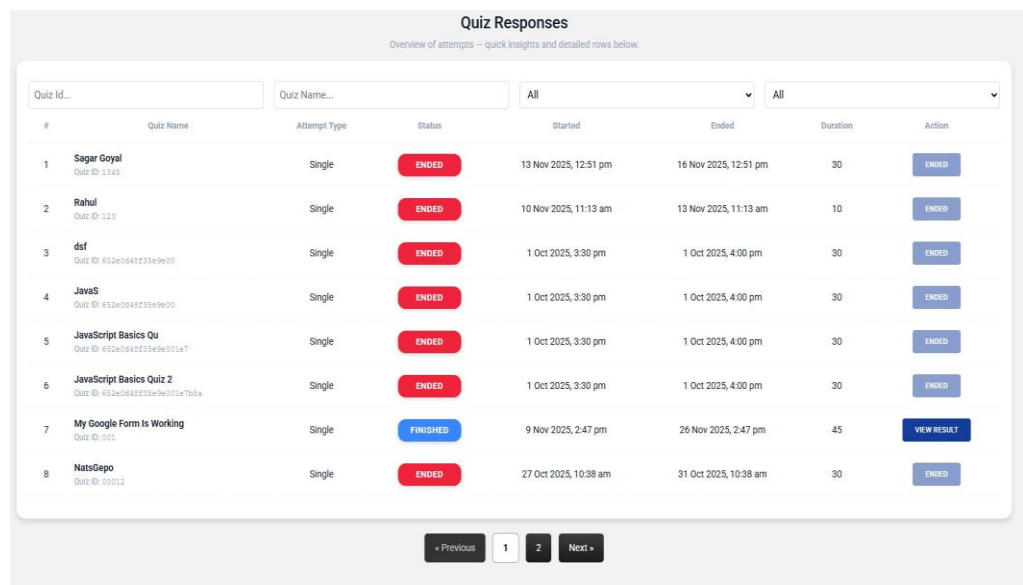


Figure 5.3.3 Quiz Listing Page

4. Quiz Interface (Question Page)

The screenshot displays a quiz interface. At the top, a header bar contains a 'P' icon, 'Test Session', 'Question 1 of 1', 'Time Remaining: 00:51:22', 'Help', and 'Finish' buttons. On the left, a 'Questions' sidebar shows a list with '1' selected. The main area contains a question: 'Which of the following is a JavaScript data type?'. Below the question are four radio button options: 'String', 'Boolean', 'Undefined', and 'All of the above'. At the bottom, there are three buttons: 'Previous', 'Clear Answer', and 'Next'.

Figure 5.3.4 Quiz Interface (Question Page)

5. Quiz Result Page

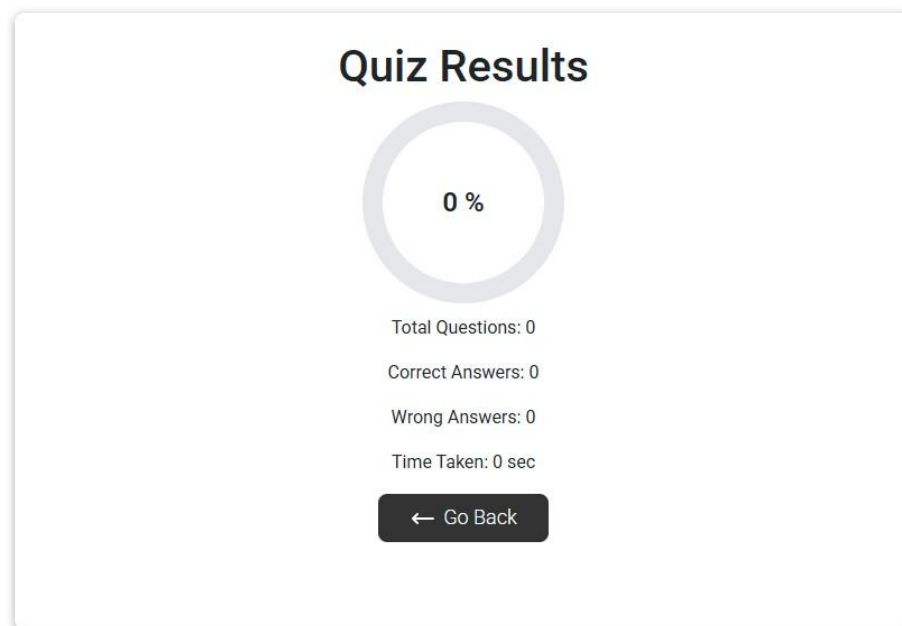


Figure 5.3.5 Quiz Result Page

6. Analytics / Performance Graphs

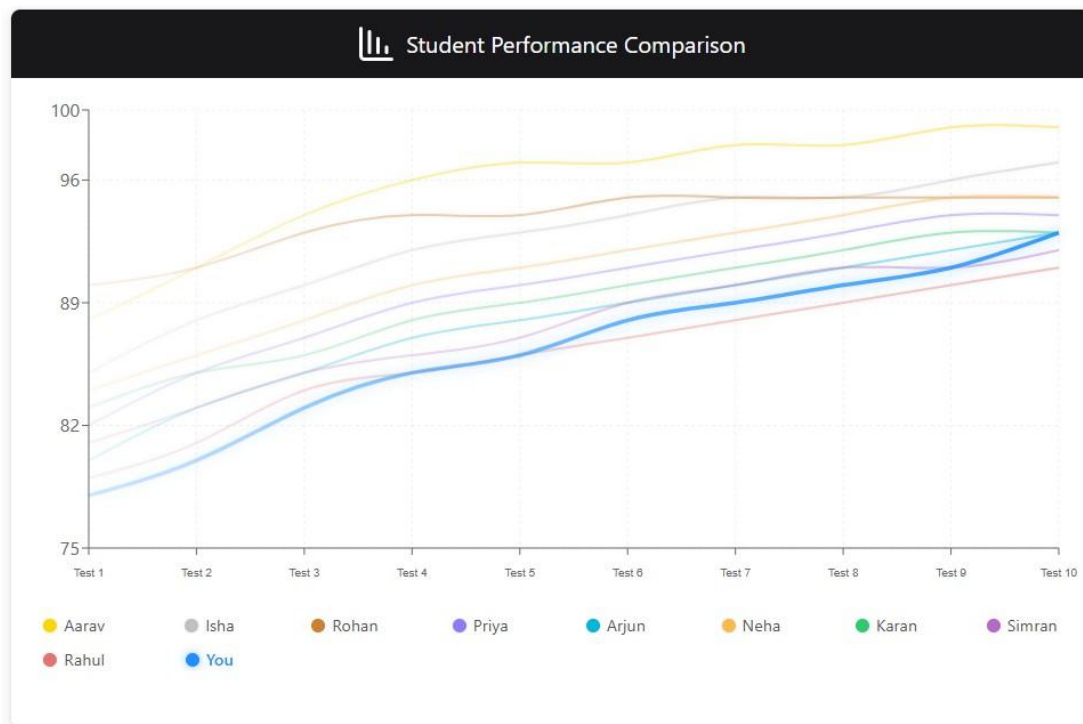


Figure 5.3.6 Performance Graph

7. Admin Dashboard

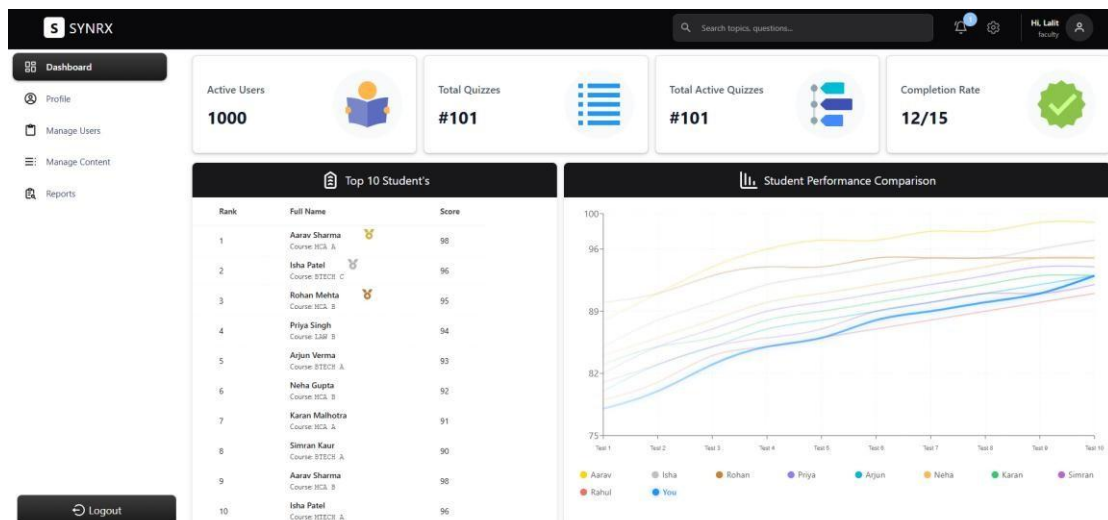


Figure 5.3.7 Admin Dashboard

10. Manage Students / Manage Faculty Page

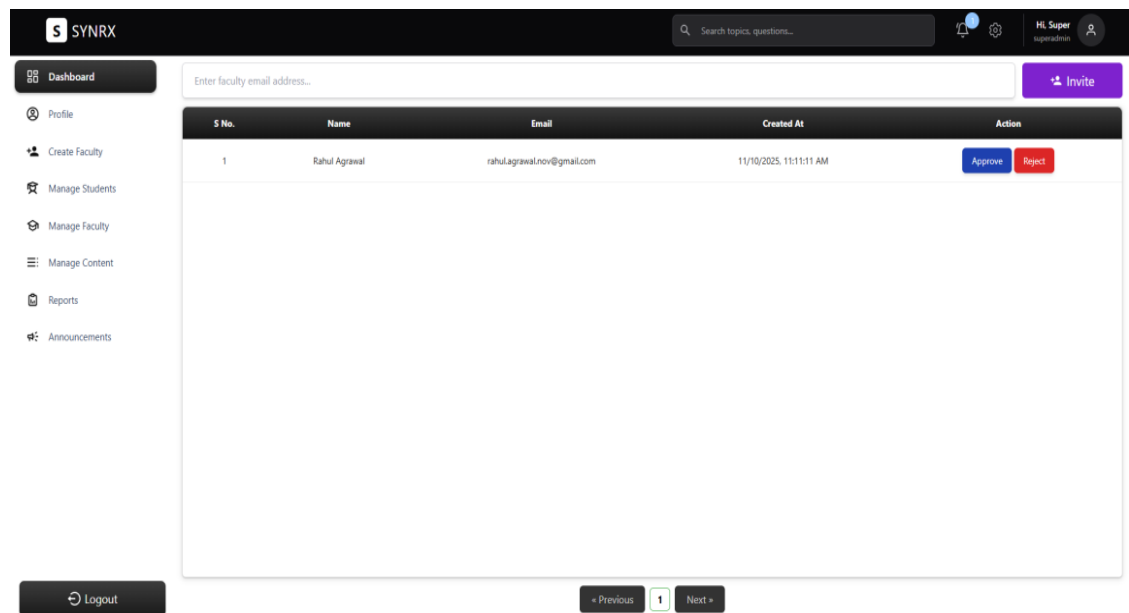


Figure 5.3.10 Manage Faculty Page

11. Voice Training Page

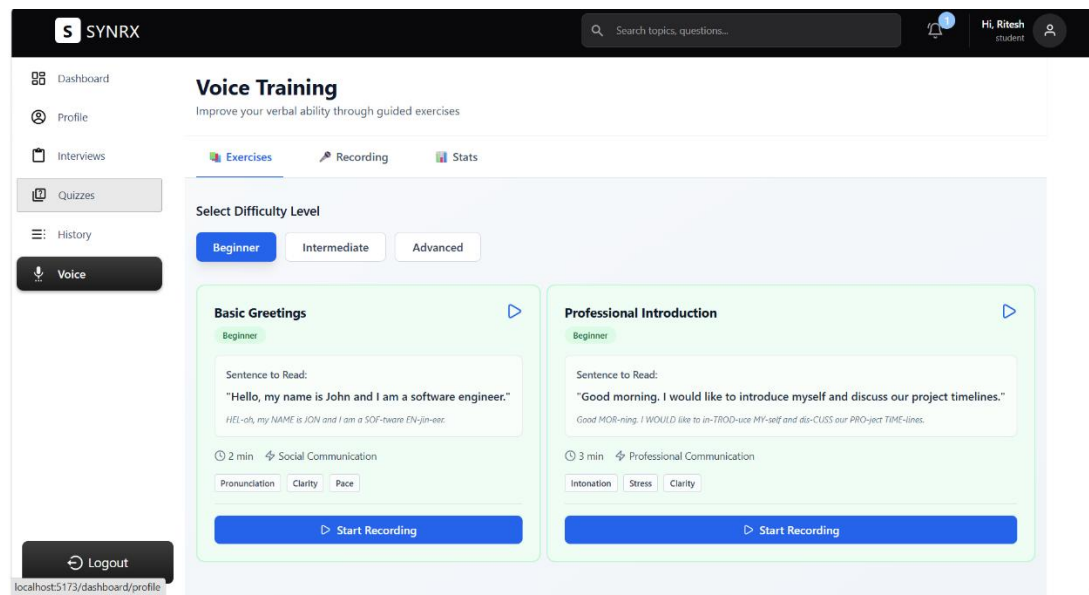


Figure 5.3.11 Voice Training Page

12. Manage User Page

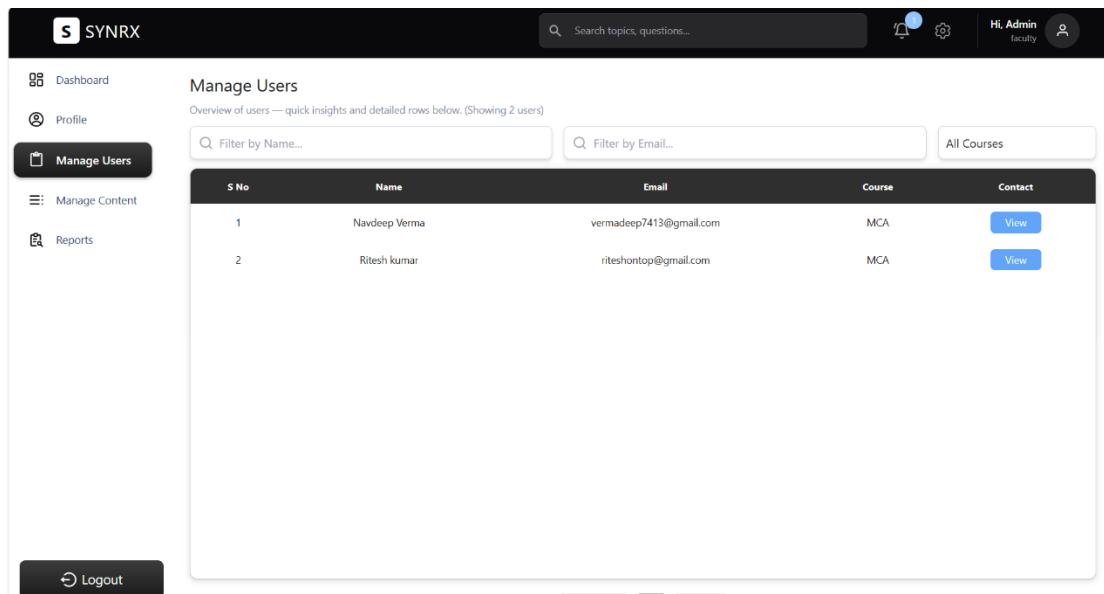


Figure 5.3.12 Manage Users Page

13. Manage Content

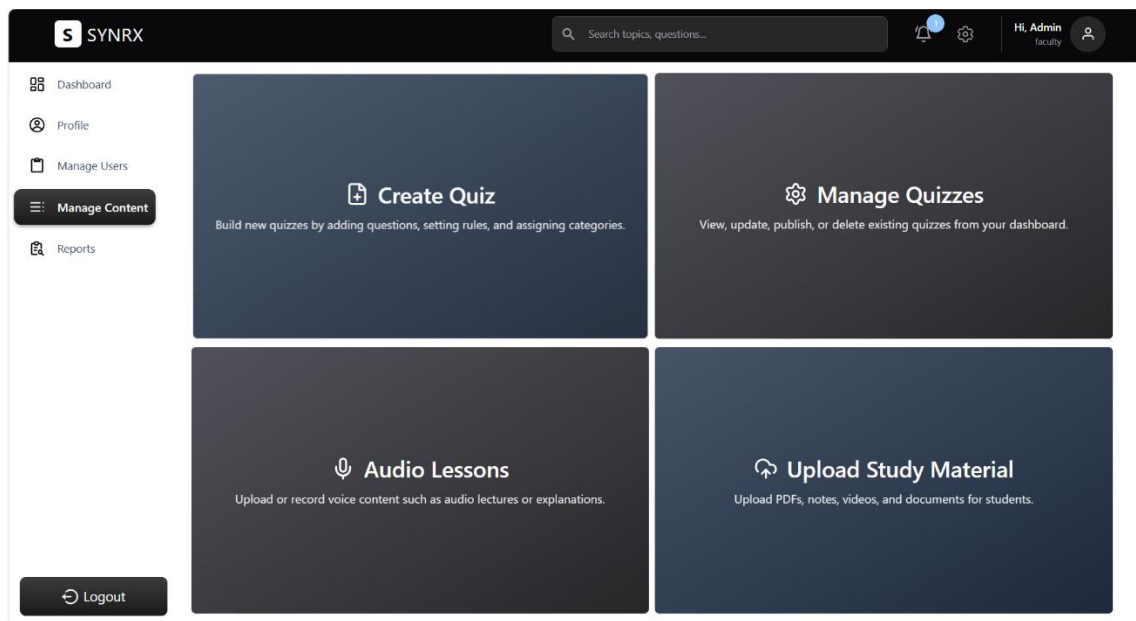


Figure 5.3.13 Manage Content Page

14. Create Faculty

SYNRRX Search topics, questions... Hi, Super superadmin

Create Faculty
Add a new faculty member to the system

Full Name
Enter full name

Email
Enter email address

Phone Number
Enter 10-digit phone number

Department **Specialization**
Select Department e.g., AI, Web Developmen

Password
Enter password

Confirm Password
Confirm password

✓ Create Faculty Clear

Dashboard
Profile
Create Faculty
Manage Students
Manage Faculty
Manage Content
Reports
Announcements
Logout

Figure 5.3.14 Create Faculty

Tested for:

- Password encryption
- Unauthorized access attempts
- Role-based route restrictions (Student/Faculty/SuperAdmin)
- Token expiration and refresh

Chapter 6

Conclusion

The Placement Preparation System SYNRX successfully fulfils the objective of providing a centralized, efficient, and user-friendly platform for students preparing for campus recruitment. By integrating study materials, quizzes, analytics, and job alerts, the system significantly enhances the preparation experience and reduces dependency on multiple external resources.

The Student Module provides a seamless environment for learning and self-assessment, enabling students to improve their problem-solving skills. The Faculty Module ensures that mentors can easily upload content, create quizzes, and monitor performance, contributing to a structured preparation process. The SuperAdmin Module maintains complete administrative control, ensuring security, proper user management, and smooth system operation.

Overall, the system meets all major functional requirements such as authentication, quiz management, performance analytics, and content delivery. It also satisfies non-functional requirements like reliability, usability, maintainability, and security. The project demonstrates how technology can simplify and streamline placement preparation in an academic environment.

Chapter 7

Summary

This project aimed to create a comprehensive Placement Preparation System to assist students in enhancing their aptitude, technical knowledge, and interview readiness. The system brings together essential features such as quizzes, study materials, job alerts, and performance tracking into a single platform.

The report began with an introduction to the problem statement, objectives, and motivation behind the system. Chapter 2 discussed the requirement analysis, module descriptions, and UML/DFD diagrams that shaped the system's architecture. Chapter 3 explained the design methodology and system diagrams, while Chapter 4 detailed implementation aspects such as technologies used, backend and frontend development, and module integration.

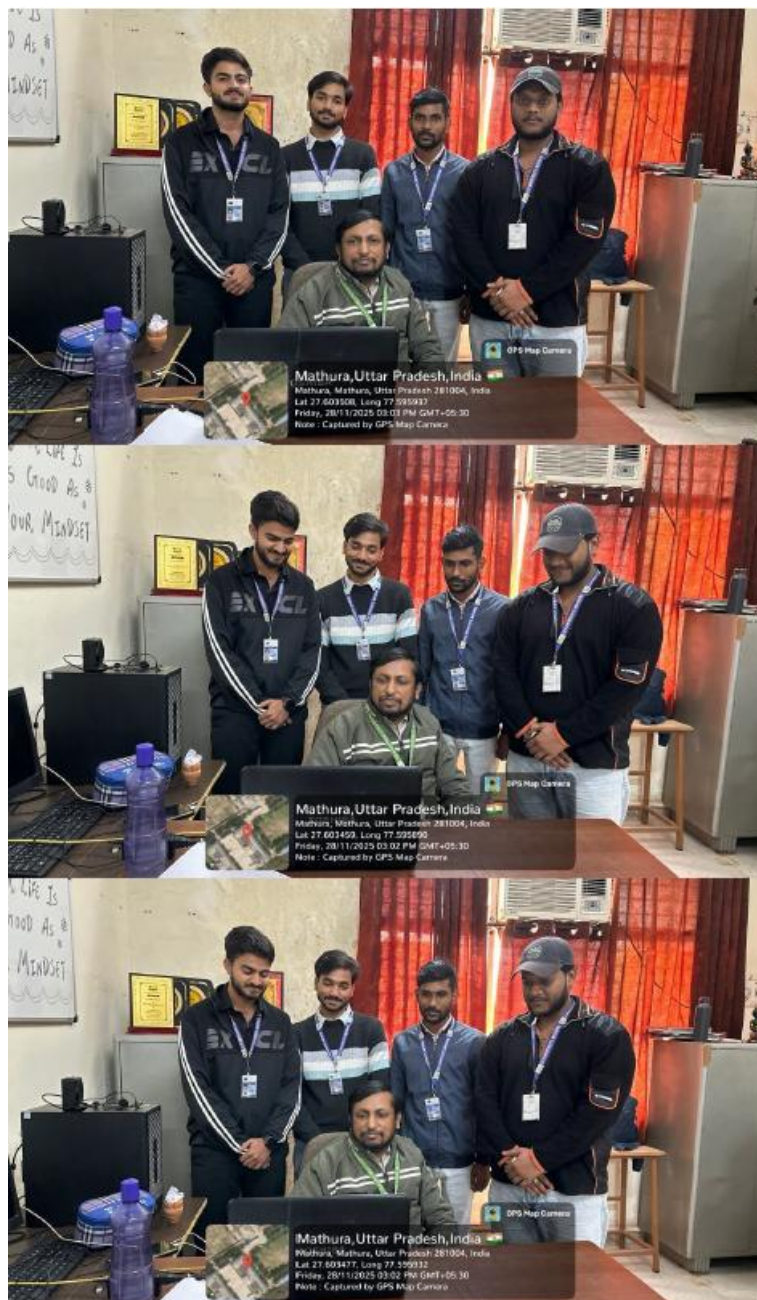
In Chapter 5, various testing techniques were applied to ensure system stability and correctness, with the results showing successful execution of all key functionalities.

The system provides value to students, faculty, and administrators alike. Students benefit from personalized analytics and organized content, faculty members gain easy tools for contributing materials and assessments, and administrators maintain oversight of the entire system.

The project showcases a reliable, scalable, and effective solution for placement training, with potential for further improvements and enhancements.

Appendix

Geo-tagged photographs with project advisor



Appendix

